

华东理工大学《数据库原理》

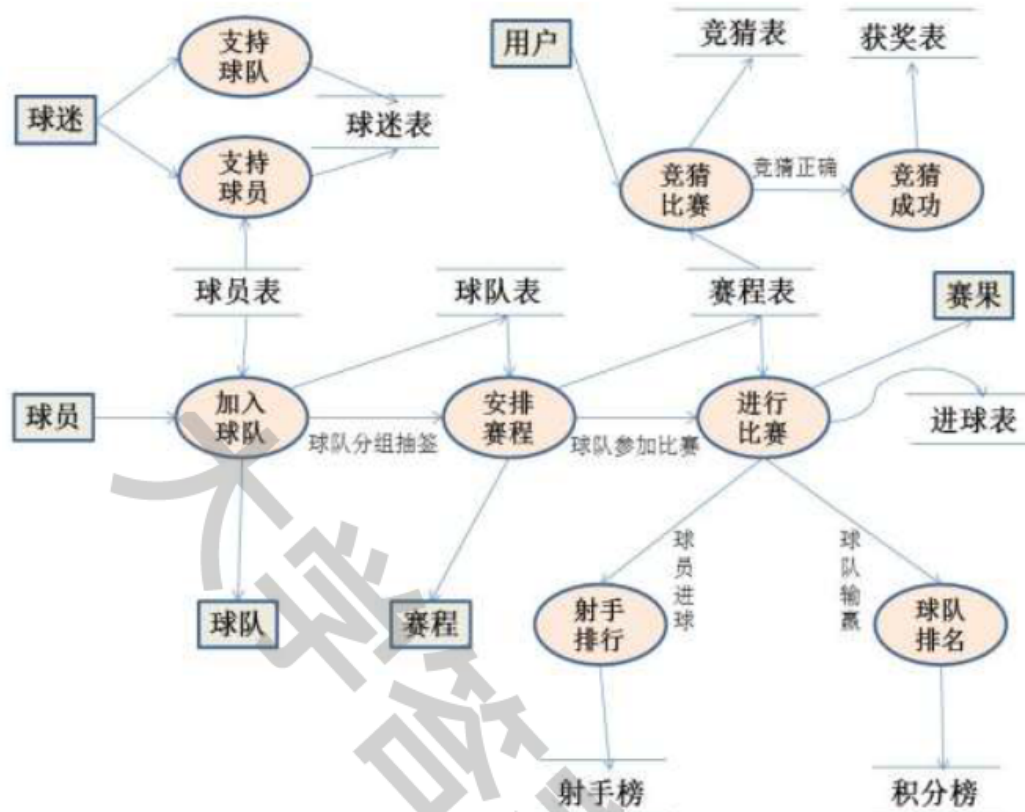
2012-2013 学年第一学期实验报告

实验名称：2012-2013 赛季欧冠联赛小组赛管理系统	实验地点：信息楼 215
所使用的工具软件及环境：SQL Server 2008、Visual Studio 2010	
一、实验目的： 1. 通过实践，掌握数据库设计方法； 2. 学会在一个实际的 RDBMS 软件平台上创建数据库系统； 3. 掌握通过 ODBC、ADO 访问数据库的方法；	
二、实验内容： 1 系统定义 项目来源：在足球比赛中，人们往往希望较容易的获得球队、球员和赛程的一些信息，而管理员则希望简单方便的对其信息进行各种管理操作，同时，也需要较容易的获得球队积分，球员射手排行和各种不容错过的进球信息，所以我们选择完成一个足球联赛管理系统，这里我们就以本年度欧洲冠军联赛为载体，旨在对其球员、球队、赛程和各种排行信息进行总结和管理，方便用户和管理员进行数据的输入和查询，同时，我们也将把球迷的投票和投票结果进行管理，然后持续更新球迷最喜爱的球队和最喜爱的球员的信息，来和用户进行简单的互动，体现出我们项目的完整性和新颖性。 项目名称：2012-2013赛季欧洲冠军联赛小组赛管理系统 项目介绍：该系统以欧冠比赛为载体，通过SQL Server 2008实现其数据库的编写和实现，利用Visual Studio 2010的C#语言完成其编写过程，对本年度欧冠联赛的各种信息进行管理，通过制作的专门界面，可以对球队、球员、比赛情况、各种统计信息进行管理。 功能分析：该系统是2012-2013赛季欧冠联赛小组赛管理系统，主要是通过欧冠联赛对足球比赛的球员、球队和比赛相关的一些活动进行系统的管理和维护，所以应该以小组赛况信息为中心，组织相关程序结构。该系统的具体功能主要分为以下几个方面： 1) 球队信息管理：显示比赛球队的大体信息，包括球队在欧冠中的分组情况，有对球队信息的增删改查的功能。 2) 球员信息管理：显示球员自身的一些信息以及与球队的关联，具有对球员信息的增删改查的功能。 3) 赛程信息管理：显示本赛季安排的比赛信息，具有增删改查的功能。 4) 比赛结果信息管理：显示比赛结果的相关统计信息，包括比赛客队和主队的比赛结果和球员的进球统计等，具有增删改查的功能。 5) 比赛积分信息管理：根据比赛结果中相关数据计算每个球队的胜场数和负场数，可以选择所在的小组，然后按照结果显示相关排名，导出积分表格，显示比赛结果的排名信息，具有增删改查的功能。 6) 球迷投票信息管理：主要记录每个登陆的球迷的基本信息和其所喜爱的球队和球员，对所得票数进行统计，得出最后的TOP排行榜，具有查询功能。 7) 竞猜管理信息系统：主要是先从赛程表中选择出并未出结果的比赛场次及其时间，然后选择用户需要竞猜的场次，填入用户名和密码以及其竞猜的比分，最后可以查询竞	

猜是否成功，以及查看获奖情况。

2 需求分析 (DFD+DD)

(1) 数据流图 (DFD)



(2) 数据字典

数据项是数据库的关系中不可再分的数据单位。下表分别列出了数据的名称、数据的类型、长度、取值能否为空。利用SQL Server2008建立“Soccer”数据库，其基本表清单及结构描述如下：

数据库中用到的表：

数据库表名	关系模式名称	备注
Course	赛程	赛程基本信息表
Player	球员	球员基本信息表
Score	积分	比赛积分信息表
Team	球队	球队基本信息表
Goal	进球	进球基本信息表
GoalScore	射手	球员进球排行表
Fans	球迷	球迷投票信息表
Prediction	竞猜	用户竞猜信息表
Award	获奖	用户获奖信息表

Course基本情况数据表，结构如下：

字段名	字段类型	允许Null值	说明
Host	varchar(20)	Not Null	主队
Guest	varchar(20)	Not Null	客队
Groupno	varchar(20)	Not Null	小组号

Gametime	datetime	Not Null	比赛时间
Hscore	int		主队得分
Gscore	int		客队得分

Player基本情况数据表，结构如下：

字段名	字段类型	允许Null值	说明
Pname	varchar(20)	Not Null	姓名
Tname	varchar(20)	Not Null	球队名称
Number	int	Not Null	球衣号码
Position	varchar(6)	Not Null	位置
Birth	datetime		生日

Team基本情况信息表，结构如下：

字段名	字段类型	允许Null值	说明
Tname	varchar(20)	Not Null	球队名称
Home	varchar(30)		主场地
Coach	varchar(20)		教练
Groupno	varchar(2)		小组号
Captain	varchar(20)		队长

Score基本情况信息表，结构如下：

字段名	字段类型	允许Null值	说明
Tname	varchar(20)	Not Null	球队名称
Won	int		胜场
Even	int		平场
Beaten	int		负场
Goalno	int		总进球数
Lostno	int		总失球数
Point	int		积分

Goal基本情况信息表，结构如下：

字段名	字段类型	允许Null值	说明
Tname	varchar(20)	Not Null	球队名称
Number	int	Not Null	球衣号码
Turn	int	Not Null	比赛轮次
Goalttime	int	Not Null	进球时间

GoalScore基本情况信息表，结构如下：

字段名	字段类型	允许Null值	说明
Place	int	Not Null	排名
Pname	varchar(20)	Not Null	姓名
Tname	varchar(20)	Not Null	球队名称
Number	int	Not Null	球衣号码

Goals	in	Not Null	进球总数
-------	----	----------	------

Fans基本情况信息表，结构如下：

字段名	字段类型	允许Null值	说明
Sno	varchar(20)	Not Null	球迷编号
Sname	varchar(20)		球迷姓名
Ssex	varc a (20)		球迷性别
Fteam	varchar(20)		球迷最喜爱球队
Fplayer	varchar(20)		球迷最喜爱队员

Prediction基本情况信息表，结构如下：

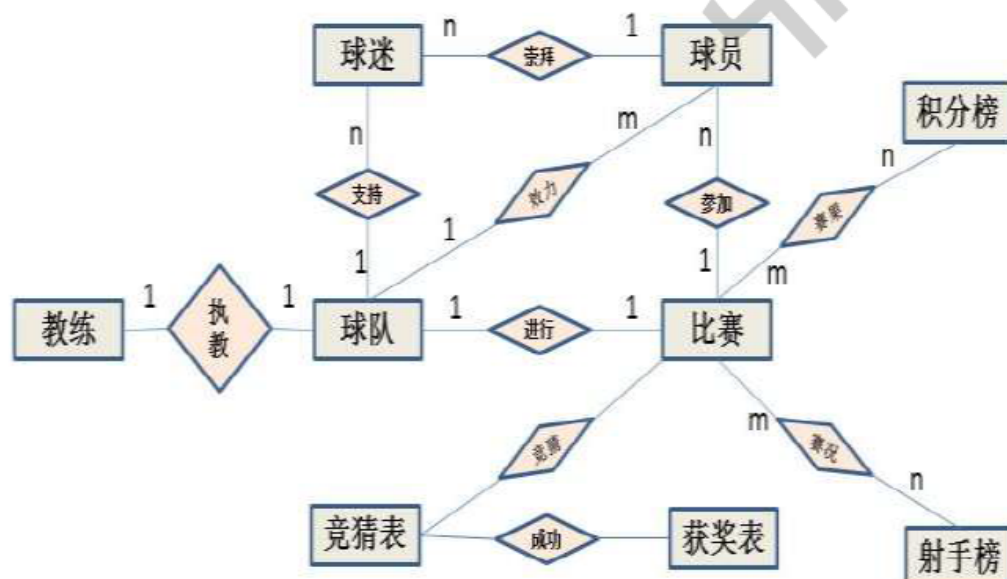
字段名	字段类型	允许Null值	说明
Users	varchar(30)	Not Null	用户名
Password	varchar(30)	Not Null	密码
Host	varchar(30)	Not Null	主队名称
Guest	varchar(30)	Not Null	客队名称
Phscore	int		竞猜主队比分
Pgscore	int		竞猜客队比分

Award基本情况信息表，结构如下：

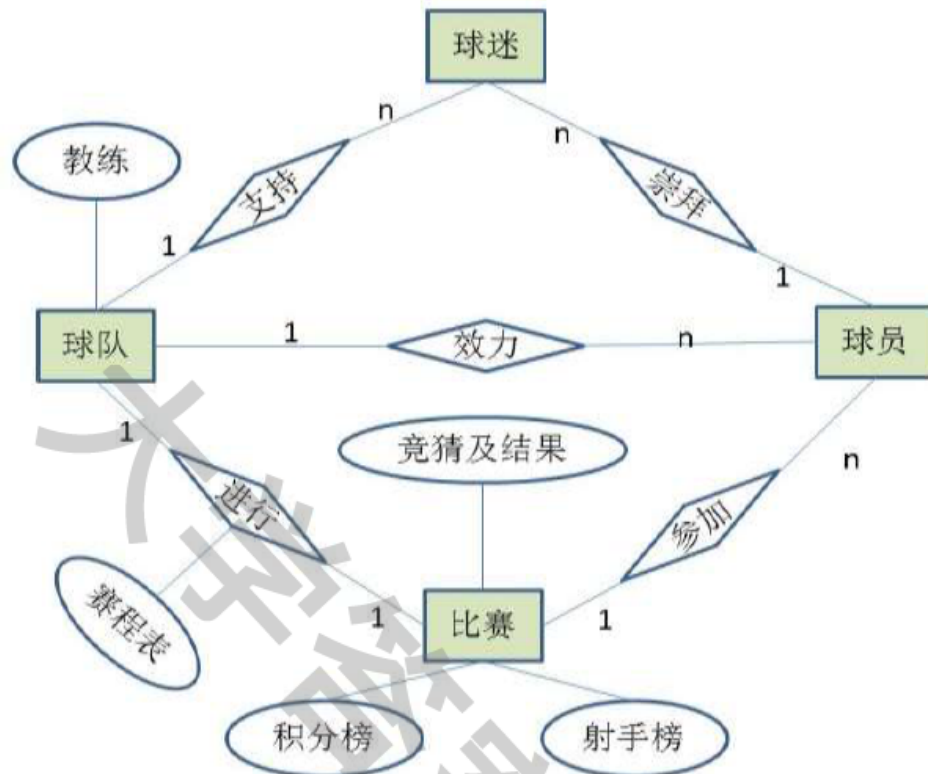
字段名	字段类型	允许Null值	说明
Name	varchar(30)	Not Null	获奖者姓名
ID	varchar(30)	Not Null	身份证号
Mail	varchar(30)	Not Null	电子邮箱
Telephone	varchar(30)	Not Null	电话号码

3 概念结构（E-R图）

由需求分析的结果，画出基本E-R如下：



对基本E-R图进行合并和消除数据冗余后得到E-R图如下：



由E-R图可知，本系统设计的实体包括：

- (1) 球员基本信息：球员姓名、所在球队名称、球衣号码、所在球队位置、生日；
- (2) 球队基本信息：球队名称、主场、教练、小组号、队长；
- (3) 赛程基本信息：主客队名、小组号、比赛日期、主客队比分；
- (4) 进球表基本信息：球队名称、球衣号码、比赛轮次、进球时间；
- (5) 积分榜基本信息：球队名称、胜场、平场、负场、总进球、总失球、积分；
- (6) 射手榜基本信息：名次、球员姓名、球队名称、球衣号码、进球数；
- (7) 球迷投票基本信息：编号、姓名、最喜爱的球队、最喜爱的球员；
- (8) 比赛竞猜基本信息：用户名、密码、主队名、客队名、竞猜主队进球数、竞猜客队进球数；
- (9) 获奖基本信息：姓名、身份证号、电子邮箱、电话号码

4 逻辑结构（关系模式集+视图）

1) 有E-R图转化而得到的关系模式如下：

- (1) Team (Tname, Home, Coach, Groupno, Captain) 其主键为Tname
 $F = \{Tname \rightarrow Home, Tname \rightarrow Coach, Tname \rightarrow Groupno, Tname \rightarrow Captain\}$
 为BC范式；
- (2) Player (Pname, Tname, Number, Position, Birth) 其主键为 (Tname, Number)
 $F = \{(Tname, Number) \rightarrow Pname, (Tname, Number) \rightarrow Position, (Tname, Number) \rightarrow Birth\}$
 为BC范式；
- (3) Course (Host, Guest, Groupno, Gametime, Hscore, Gscore) 其主键为 (Host, Guest)

$F=\{(Host,Guest)\rightarrow Groupno,(Host,Guest)\rightarrow Gametime,(Host,Guest)\rightarrow Hscore,(Host,Guest)\rightarrow Gscore\}$
 为BC范式;
 (4) Goal (Tname,Number,Turn,Goaltime) 其主键为 (Tname,Number,Turn,Goaltime)
 $F=\{(Tname,Number,Turn,Goaltime)\}$
 为BC范式
 (5) Score (Tname,Won,Even,Beaten,Goalno,Lostno,Point) 其主键为Tname
 $F=\{Tname\rightarrow Won, Tname\rightarrow Even, Tname\rightarrow Beaten, Tname\rightarrow Goalno, Tname\rightarrow Lostno, Tname\rightarrow Point\}$
 为BC范式;
 (6) GoalScore (Place,Pname,Tname,Number,Goals) 其主键为 (Tname,Number)
 $F=\{(Tname,Number)\rightarrow Place, (Tname,Number)\rightarrow Pname, (Tname,Number)\rightarrow Goals\}$
 为BC范式;
 (7) Fans (Sno,Sname,Ssex,Fteam,Fplayer) 其主键为Sno
 $F=\{Sno\rightarrow Sname, Sno\rightarrow Ssex, Sno\rightarrow Fteam, Sno\rightarrow Fplayer\}$
 为BC范式;
 (8) Prediction (Users>Password,Host,Guest,Phscore,Pgscore) 其主键为 (User>Password,Host,Guest)
 $F=\{(User>Password,Host,Guest)\rightarrow Phscore, (User>Password,Host,Guest)\rightarrow Pgscore\}$
 为BC范式;
 (9) Award (Name,ID,Mail,Telephone) 其主键为ID
 $F=\{ID\rightarrow Name, ID\rightarrow Mail, ID\rightarrow Telephone\}$
 为BC范式。

2) 确定关系模型的存取方法

在将概念模型转换成物理模型之后，我们可以对物理模型进行设计，双击物理模型的关系，可以对该关系的名称、注释等信息进行查询。可对该关系的属性列进行设计，可分别设置其名称、码、数据类型以及主码、是否为空等。在实际设计中最常用的存取方法是索引，使用索引可以大大减少数据的查询时间，在建立索引时应遵循：在经常需要搜索的列上建立索引；在主关键字上建立索引；在经常用于连接的列上建立索引，即在外键上建立索引；在经常需要根据范围进行搜索的列上创建索引，因为索引已经排序，其指定的范围是连续的等规则。才能充分利用索引的作用避免因索引引起的负面作用。

3) 本系统用到的视图

(1) 基于Fans表，创建一个选出Fteam及其所得票数的视图s_fteam1

```

Create view s_fteam1 As
SELECT DISTINCT Fteam, COUNT(Fteam) AS number
FROM    Fans
GROUP BY Fteam
  
```

(2) 创建基于视图s_fteam1的视图s_fteam，选出Fteam排名前三的数据

```

Create view s_fteam As
SELECT    TOP 3  Fteam, number
FROM      s_fteam1
  
```

ORDER BY number DESC

(3) 基于Fans表, 创建一个选出Fplayer及其所得票数的视图s_fplayer1

Create view s_fplayer1 As

SELECT DISTINCT Fplayer, COUNT(Fplayer) AS number

FROM Fans

GROUP BY Fplayer

(4) 创建基于视图s_fplayer1的视图s_fplayer, 选出Fplayer排名前三的数据

Create view s_fplayer As

SELECT TOP 3 Fplayer, number

FROM s_fplayer1

ORDER BY number DESC

(5) 创建基于Course表的视图s_prediction, 选出还没有比分的比赛赛程基本信息

Create view s_prediction As

SELECT Gametime, Host, Guest

From Course

WHERE Hscore is null and Gscore is null

(6) 创建基于Prediction表的视图s_quizquery, 选出其中的除密码以外的所有信息, 密码要用户自己输入, 判断正确后才能进行下面的操作

Create view s_quizquery

Select Users, Host, Guest, Phscore, Pgscore

Form Prediction

4) 本系统用到的存储过程

(1) 根据传入参数是Tname还是Groupno来从Team表中查询其他的信息

```
Create Procedure p_soccerStock @column varchar(30), @value  
varchar(50)
```

```
AS
```

```
IF @column='Tname'
```

```
    SELECT Tname, Home, Coach, Groupno, Captain
```

```
    FROM Team
```

```
    WHERE Tname LIKE '%' + @value + '%'
```

```
ELSE
```

```
    SELECT Tname, Home, Coach, Groupno, Captain
```

```
    FROM Team
```

```
    WHERE Groupno LIKE '%' + @value + '%'
```

(2) 根据传入参数Groupno来从Team表和Score做连接, 查询其每组的球队和积分

```
Create Procedure p_scoreStock @value NVarchar(2)
```

```
AS
```

```
    SELECT Score.Tname, Won, Even, Beaten, Goalno, Lostno, Point
```

```
    FROM Team, Score
```

```
    WHERE Team.Tname=Score.Tname and Groupno LIKE '%' + @value + '%'
```

(3) 根据传入参数Tname, 从Player表中查询属于某球队的所有球员的信息

```
Create Procedure p_playerStock @column varchar(20), @value
```

```
varchar(50)
```

```

AS
    SELECT Pname,Number,Position,Birth
    FROM Player
    WHERE Tname LIKE '%' + @value + '%'

```

(4) 根据传入参数Turn,从Goal表中查询某轮比赛的所有球员进球信息

```

Create Procedure p_bookStock] @column varchar(30),@value
varchar(50)
AS
    SELECT Tname,Number,GoalTime
    FROM Goal
    WHERE Turn LIKE '%' + @value + '%'

```

5 物理设计（存储结构及索引）

确定数据库的存储结构主要指确定数据的存放位置和存储结构，包括确定关系、索引、日志、备份等的存储安排及存储结构，以及确定系统存储参数的配置。

索引设计：各表都有自己的主键，主键为聚集索引，而且每个表只能建立一个聚集索引，所以只能创建非聚集索引。

(1) 创建Course信息表代码：

```

CREATE TABLE Course(
    Host varchar(20) NOT NULL,
    Guest varchar(20) NOT NULL,
    Groupno varchar(2) NOT NULL,
    Gametime datetime NOT NULL,
    Hscore int NULL,
    Gscore int NULL,
    CONSTRAINT PK_Course PRIMARY KEY (Host,Guest)
)

```

(2) 创建球队Team信息表、球员Player信息表、Goal信息表、Score信息表、GoalScore信息表、Fans信息表、Prediction信息表、Award信息表代码，在此不再赘述。

6 系统实现（代码见附录）

1) 主界面

功能：主界面的主要功能是对用户选择相应操作的触发，如球队管理功能、球员管理功能等。

界面设计：



2) 球队管理模块

功能：添加、删除、修改和查询球队的信息

界面设计：



3) 球员管理模块

功能设计：添加、删除、修改和查询球员的信息

界面设计：



4) 赛程管理模块

功能设计：分别对赛程表和进球表进行查询等操作。界面设计：

进球表

请输入比赛轮次: 3 进球查询

	球队名称	球衣号码	进球时间
	马拉加	7	64
	曼城	8	22
	曼联	6	62
	曼联	14	25
	曼联	14	75

添加 退出

赛程表

1 / 96

主队	客队	小组号	比赛时间	主队进球数	客队进球数
AC米兰	安德莱赫特	C	2012/9/19	0	0
奥林匹亚科斯	沙尔克04	B	2012/9/19	2	1
巴黎圣日尔曼	基辅迪纳摩	A	2012/9/19	4	1
多特蒙德	阿贾克斯	D	2012/9/19	1	0
皇家马德里	曼城	D	2012/9/19	3	2
马拉加	泽尼特	C	2012/9/19	3	0
蒙彼利埃	阿森纳	B	2012/9/19	1	2
萨姆索布洛姆	波尔多	A	2012/9/19	0	2

主队: AC米兰
客队: 安德莱赫特
Groupno: C
GameTime: 2012年 9月19日
Home: 0
Score: 0

5) TOP 榜管理模块

功能设计: 主要是对小组积分和球员射手榜的查询等功能

界面设计:

积分榜

添加 查询

请输入小组号: B 查询

	球队名称	胜场	平场	负场	进球数
	阿森纳	9	0	7	0
	奥林匹亚科斯	2	0	2	7
	蒙彼利埃	0	1	3	5
	沙尔克04	2	2	0	8
*					

修改 关闭

射手榜

射手排行榜:

	rowState	Place	Pname	Tname	Number	Goals
	Unchanged	1	C罗纳尔多	皇家马德里	7	5
	Unchanged	2	亨特拉尔	沙尔克04	25	4
	Unchanged	3	索尔达多	瓦伦西亚	9	4
	Unchanged	4	奥斯卡	切尔西	11	4
	Unchanged	5	范佩西	曼联	20	3

名次: 球员号码:

球员姓名: 进球数:

球队名称: 状态:

6) 球迷投票管理模块

功能设计: 主要是记录球迷的基本信息和对投票的记录与管理
界面设计:

投票结果

Top3

	最喜爱的球队	票数
▶	阿森纳	5
	皇家马德里	2
	巴塞罗那	2
*		

	最喜爱的球员	票数
▶	梅西	5
	范佩西	2
	鲁尼	1
*		

关闭

球迷投票

学号:

姓名:

性别:

最喜欢的球队:

最喜欢的球员:

确认投票 投票结果查询

7) 用户竞猜管理模块

功能设计: 主要是记录用户的基本信息和对比赛结果的竞猜情况,

界面设计:

比赛竞猜

比赛时间	主队名称	客队名称
2012/10/5	AC米兰	泽尼特
2012/11/22	阿贾克斯	多特蒙德
2012/11/22	阿森纳	蒙彼利埃
2012/11/22	安德莱赫特	AC米兰
2012/12/5	奥林匹亚科斯	阿森纳

主队名称: AC米兰 客队名称: 泽尼特

用户登陆与竞猜

用户名: Iverson 竞猜主队进球数: 1

密码: 竞猜客队进球数: 1

确认竞猜 关闭

竞猜查询

用户名	主队名	客队名	竞猜主队进球数	竞猜客队进球数
Iverson	AC米兰	泽尼特	1	1

用户登陆

用户名: Iverson

请输入密码:

用户操作

竞猜结果查询 关闭



三、出现的问题和解决方案

1) 关于界面:

(1) 刚开始编程的时候对按钮的操作却没有任何反应, 很纠结, 后来发现是按钮的Name属性与函数编程中的按钮的名称弄混了, 比如有个按钮的Name为button1, 则必须该按钮的Click函数为 `private void button1_Click(object sender, EventArgs e)`, 否则用户单击的时候将不会产生任何响应。

(2) 对界面的安排和设置不合理, 导致界面很乱, 许多静态文本和按钮等混淆在一起, 最终导致功能完善的不合理。最后查阅资料, 了解到可以在Form中插入多个Panel面板或者是GroupBox分组框, 将每个不同功能的控件, 包括Label、TextBox、Button、RadioButton

ComboBox等放在相同的Panel或者GroupBox中, 这样界面分配合理且有条理, 不仅美观, 对功能的实现也起到了一点帮助的作用。

(3) 对ComboBox内的选项进行初始化的时候遇到了一些问题, 因为我把Form的Load函数初始化Combox和在Combox中的Item初始化的时候都做了一遍, 不仅浪费了代码和时间, 有时还会导致冲突错误。解决方法很简单, 两种方法取其一种就可以了, 这里推荐直接在Item中添加选项, 因为简单和全可视化操作, 但是如果其中的选项要在数据库的表中查询, 则必须使用Load方法的初始化其选项。

(4) 实验完成后发现窗口的背景和所有控件的大小都不随窗口的大小改变而改变, 即每当最大化窗口或者拉伸窗口的时候, 控件并不随着改变其大小, 这样就会导致界面不美观。查了一下资料, 有两种解决方案: 第一, 更改控件的Dock或Anchor的属性, 使其可以随着窗口的变换而变换, 即将Anchor设置为Top, Left, Bottom, Right或者将Dock设置为靠近上下左右四个方向停靠, 试验了一下, 果真可以, 但是当窗口中控件很多的情况下, 不仅一个一个设置很麻烦, 而且当Anchor和Dock属性用多了以后, 窗口拉伸的时候会出现不停的闪烁的现象, 而且无法避免; 第二, 直接设置完成控件的大小后, 设置其大小不可改变, 这样只需要更改窗口的MaximizeBox为False, 并将FormBorderStyle设置为FixedSingle即可, 这样既方便又有效限制了用户的操作规范, 不至于发生什么界面混乱的状况, 但是不能随意修改大小某些时候也会有些限制; 所以, 这里我们仅采用第二种做法。

2) 关于数据库:

(1) 往表中插入数据, 只能每次插入一条记录吗?

最终答案：是的，在 SQL Server 中，用 INSERT ... VALUES... 每次只能插入一条记录。

(2) 对定义了外键的表插入数据时，例如对 Reports 表插入元组时，总是说插入错误。

最终解决：在表中定义了外键之后，对表进行插入时，要注意数据与被引用表中的一致性。要插入的元组中，属于外键的数据必须是在被引用表中存在的。

例如：在 Reports 中定义 Sno 为外键，参照表 Students 中的 Sno。那么在 Reports 表中要插入 ('S88', 'T02', 'C01', 88) 时，如果 Students 表中没有 "S88" 这条记录，那么插入失败。另外，如果这些规定都符合了，但插入语句 VALUES () 括号里的各字段值顺序跟表中的顺序不一致，也会出现同样的错误提示。

(3) 在 SQL 语句的末尾都可以加分号 ";"，但是为什么在创建视图的语句后面加了分号会报错？

最终解决：在 SQL Server 2000 中，查询分析器中的语句规则是，各条语句后面不用加任何标点符号。但鉴于人们的使用习惯，基本上允许在每条语句的后面加上 ";"。个别语句可能会有影响。另外也可以用 "GO" 来隔开各条语句。它的意义在于告诉系统，在两个 "GO" 之间的语句作为一批去处理。

(4) 为何建了新的用户，却登录不了？

最终解决：建立管理员账号（如 sa）时，若定义为以 WINDOWS 验证登录。而建立其它用户，如 USER1 时，定义为 WINDOWS+SQL 验证登录。这样 USER1 无法登录。只有将 sa 登录方式改为以 WINDOWS+SQL 验证登录之后，方可以 USER1 登录。

若不改 SA 的登录方式，而是建立 USER1 时，将其登录方式设为以 WINDOWS 验证登录，是不行的。因为若数据库所有者设为以 WINDOWS 登录后，再建立新用户时，只能从 WINDOWS 的用户里面选择，也就是选择在 WINDOWS 系统域内的用户名。

(5) 在 SQL 语句的末尾都可以加分号 ";"，但是为什么在创建视图的语句后面加了分号会报错？

最终解决：在 SQL Server 2000 中，查询分析器中的语句规则是，各条语句后面不用加任何标点符号。但鉴于人们的使用习惯，基本上允许在每条语句的后面加上 ";"。个别语句可能会有影响。另外也可以用 "GO" 来隔开各条语句。它的意义在于告诉系统，在两个 "GO" 之间的语句作为一批去处理。

3) 关于二者连接

(1) 首先刚开始做的时候，用的是 VS2010 和 SQL Server 2000，但是在建立数据库连接的时候发现 VS2010 上显示，不支持 SQL Server 2000 的版本，所以导致无法建立连接。

解决方法：干净彻底的卸载 SQL Server 2000，并重新安装 SQL Server 2008，这个虽然简单，但是我却重装了三次，花了将近半天的时间，最主要的是一定要将来 SQL Server 2000 或其他版本的文件彻底的删除干净，包括注册表，一点都不能残留，不然每次安装将无法成功。

(2) 刚开始在编写程序的时候，每当创建连接的时候，使用 SqlConnection, SqlCommand 等其他 ADO 中的类建立连接的时候，发现编译的时候不能通过，错误显示没有导入相应的命名空间，由于刚开始接触，还是不太懂，所以上网找了一下资料，才恍然大悟，原来需要导入 System.Data.SqlClient, 不然无法使用其中的类和方法。

解决方法：第一，直接在代码的开始部分写上 using System.Data.SqlClient, 这种方法虽然简单，但是在每个 .cs 文件中都必须写一次，所以比较麻烦；第二，在 VS 解决方案资源管理器窗口中右键选择 '引用'，选择 '添加引用'，然后在 .Net 中选择需要的引用 System.Data.SqlClient, 这样以后就不用每次都添加了，同样如需添加其他的引用，都使用这种方法。

(3) 使用 ADO.NET 中的 DataSet 类时也出现了一些问题, 比如使用其对表进行修改, 删除和添加, 分明显示已经成功, 但是在数据库中却看不见信息的变换, 导致以后再操作时出现错误。这个问题查了一下课本, 原来 DataSet 的使用是不需要与数据库连接的, 即在缓存中存在与数据库中的表一样的表, 我们可以对其进行任何操作, 而不需要建立与数据库的连接, 这样大大简化了程序的执行步骤, 使时间可观的减少了, 加快了运行的速度, 但是必须注意的是, 这样做的修改并不会传递到数据库中去, 必须认为的调用 Update 方法来将修改保存到数据库中去。

解决方法: 待执行完需要修改数据库的程序以后, 显示的调用 Update 方法, 使修改能够传递到数据库中, 这样将会保证下次实验的数据的正确性。

(4) 在需要导出表格的时候, 每次都利用 Label 控件和 TextBox 控件的方法, 这样做很麻烦而且对其命名和使用也相当的不变, 同时, 也要时刻的建立与数据库的连接, 编写大量的代码, 在使用数据库视图和存储过程的时候也会遇到很大的困难。

解决方法: ADO.NET 中有这样一种控件, 叫做 DataGridview, 它可以操作对数据库表格, 视图, 存储过程等的所有绑定与显示, 可以大大简化程序员的工作量, 更加方便快速的完成所需要的工作。其位置在工具箱的数据栏中, 直接拖到 Form 窗口中, 然后对其进行绑定, 修改其属性, 完成所需要完成的工作。

四、收获和体会

经过几周紧张的课程设计, 我们终于完成了所选的设计题目《足球比赛管理系统》。期间的烦恼、郁闷、快乐, 纵横交错。程序编写时遇到许多问题(如上所述), 往往一个小细节的问题, 就导致整个程序的运行错误, 想来想去, 要花费好大功夫, 当问题被发现时不由你不发表感慨“细节决定成败啊!”。

此课程设计将学到的高级编程语言与数据库相结合, 设计一个更接近生活的应用软件。这不仅需要数据库知识, 还要有对高级编程语言的熟练的应用的能力。几周的实验时间, 有点紧迫, 不得不抓紧时间来完成。这就更需要耐心, 不能因一个小问题, 而延误整个程序设计开发的进展。在很短的时间, 难免有更多的考虑不周全的地方, 从而导致软件的设计缺陷。

开发数据库管理信息系统需要选择两种工具, 也就是前台开发语言和后台数据库。选择前台开发工具时, 应该考虑客户需求、系统功能和性能要求以及开发人员的习惯等。

开发该案例所需要的技术主要有以下几点: 所选开发工具的基本编程方法(我们小组选用的是 Visual Studio 2010, 选择了用 C# 开发界面); 基本的后台数据库管理的方法, 例如创建数据库、创建表和视图、备份和还原数据库等; 常用 SQL 语句的相关使用以及 ADO 数据库访问技术。

因为设计和开发数据库不仅用到了数据库的基本知识和数据库设计技术, 还用到了应用领域的知识。我们的数据库知识涉及到了欧冠联赛, 我个人在这方面不是很熟悉的, 对篮球还有一点了解, 但是足球比赛的话, 可以说是除了会踢, 其他的比赛章程、规则基本上就是个“二愣子”了, 所以在数据分析的时候感觉很吃力, 后来又自己分析了一段时间才了解了。后来的数据库设计基本知识还是觉得很容易的, 但是后来的一些数据流图、画 E-R 图等也出现了问题, 不过后来都解决了。

在界面方面, 因为个人的编程能力不是很好, 所以基本上都是小组其他成员编写的, 个人不得不感叹编程语言的神奇啊, 看来后面一段时间该补一下这方面的知识了。

经过这段时间的实验, 自己的最大体会是, 个人感觉比较适合做数据库还有 WEB 网页方面的内容, 而且兴趣也比单纯的编程兴趣大。而且, 我是个比较急性子的人, 但是像这种实验又是好多细节决定成败的, 所以也发现很多事情并不是瞎着急就做得好

的。总之，通过这次课程实验，自己感悟还是颇深的。该软件如果实际应用，会反映出更多的问题，因为软件的维护，本来就是发现问题且去解决问题，以适应环境的变化。最后，还要说几句，通过本次课程设计，我学到了更多、掌握了更多的知识，真所谓有付出才会有收获啊!!!

学号	姓名	班级	成绩	备注
		计 101		
		计 101		
		计 101		
教师签名： 日期：				

本附录以一个窗口（射手榜）的增、删、改、查为例，详细的列出了其每部分的完成所需要的代码，由于篇幅有限，其他窗口和管理功能的实现与之类似。

首先给出该窗口的界面：

rowState	Place	Pname	Tname	Number	Goals
Unchanged	1	C罗纳尔多	皇家马德里	7	5
Unchanged	2	亨特拉尔	沙尔克04	25	4
Unchanged	3	索尔达多	瓦伦西亚	9	4
Unchanged	4	奥斯卡	切尔西	11	4
Unchanged	5	范佩西	曼联	20	3

名次: 球员号码:

球员姓名: 进球数:

球队名称: 状态:

添加 删除 修改 确认 重置

按照控件的顺序，每个 TextBox 命名为 txtPlace,txtPname,txtTname,txtNumber,txtGoalno,txtState; 分别对应的是名次、球员姓名、球队名称、球员号码、进球数和状态。每个 Button 控件命名为 button1, button2, button3, button4, button5, button6; 分别对应的是添加、删除、修改、确认和重置。

步骤一：首先该窗口的名称是 ScoreList，所以其主框架代码如下，包括所有的引用和基本的构造函数等：

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Data.SqlClient;
namespace Soccer
{
    public partial class ScoreList : Form
    {
        public ScoreList()
        {
            InitializeComponent();
        }
    }
}
```



```
}
}
```

步骤二：其中由于必须建立与数据库的连接，所以先声明一些连接对象，方便以后各个函数的使用，添加如下代码到主程序中：

```
SqlConnection cn;
SqlDataAdapter da;
DataSet ds;
DataTable dt;
```

步骤三：首先当载入次窗口时，必须显示出数据库中已经有了的射手排行榜的所有信息，如图所示，其是 DataGridView 控件，命名为 dataGridView1，尚未建立与任何数据表格的绑定，所以开始的时候必须在 Load 函数中进行初始化，添加如下代码到主程序中：

```
private void ScoreList_Load(object sender, EventArgs e)
{
    // 使用 DataAdapter 对象获取数据并填充 DataSet
    String connString = "Data Source=(local);Initial Catalog=Soccer;Integrated
Security=True";
    cn = new SqlConnection(connString);
    da = new SqlDataAdapter("select * from GoalScore", cn);
    ds = new DataSet();
    try
    {
        da.Fill(ds, "GoalScore");
    }
    catch (SqlException ex)
    {
        MessageBox.Show(ex.Message);
    }
    dt = ds.Tables["GoalScore"];
    // 定义 DataGridView 的表头（列）
    dataGridView1.Columns.Add("rowState", "rowState");
    foreach (DataColumn col in dt.Columns)
        dataGridView1.Columns.Add(col.ColumnName, col.ColumnName);
    // 调用自定义函数在表格中显示数据
    FillGrid();
}
private void FillGrid()
{
    dataGridView1.Rows.Clear();
    // 根据 DataSet 行的个数在 DataGridView 中添加行
    dataGridView1.Rows.Add(dt.Rows.Count);
    // 循环从 DataSet 中取所有行和每行的所有列
    int i = 0;
    foreach (DataRow row in dt.Rows)
    {
```

```

        DataGridViewRow r1 = dataGridView1.Rows[i];
        // DataGridView 的第一列用于显示行状态
        r1.Cells[0].Value = row.RowState.ToString();
        for (int j = 0; j < dt.Columns.Count; j++)
            r1.Cells[j + 1].Value = row[j].ToString();
        i++;
    }
}

```

步骤四：当表格中出现所有射手榜信息的时候，这时如果选中某一行，则希望文本框中该项数据与之相当应，所以必须添加选中改变的函数，使表格选中行与文本框的信息同步变化，添加如下代码到主程序中去：

```

// 使表格选中行与文本框的信息同步变化
private void dataGridView1_SelectionChanged(object sender, EventArgs e)
{
    if (dataGridView1.CurrentRow.Cells[0].Value != null)
    {
        DataGridViewRow dgRow = dataGridView1.CurrentRow;
        txtPlace.Text = dgRow.Cells[1].Value.ToString();
        txtPname.Text = dgRow.Cells[2].Value.ToString();
        txtTname.Text = dgRow.Cells[3].Value.ToString();
        txtNumber.Text = dgRow.Cells[4].Value.ToString();
        txtGoals.Text = dgRow.Cells[5].Value.ToString();
    }
    else //如果没有行被选中
    {
        txtPlace.Text = "";
        txtPname.Text = "";
        txtTname.Text = "";
        txtNumber.Text = "";
        txtGoals.Text = "";
    }
}

```

步骤五：将新的射手信息填入文本框中，单击按钮‘添加’，则新添加的信息被填充到上面的表格中去，且 rowState 显示 Added，添加如下代码到主程序中去：

```

private void button1_Click(object sender, EventArgs e)
{
    // 在 DataSet 中添加新行
    DataTable dt = ds.Tables["GoalScore"];
    DataRow dr = dt.Rows.Add(txtPlace.Text, txtPname.Text, txtTname.Text,
        txtNumber.Text, txtGoals.Text);

    // 将当前新添加的行加入到 DataGridView 中以显示
    dataGridView1.Rows.Add();
}

```

```

DataGridViewRow row = dataGridView1.Rows[dt.Rows.Count-1];
row.Cells[0].Value = dr.RowState.ToString();
row.Cells[1].Value = dr[0];
row.Cells[2].Value = dr[1];
row.Cells[3].Value = dr[2];
row.Cells[4].Value = dr[3];
row.Cells[5].Value = dr[4];
}

```

步骤五：选中某一行信息，如果需要删除，则点击‘删除’按钮，则该行的 rowState 将显示 Deleted，添加如下代码到主程序中：

```

private void button2_Click(object sender, EventArgs e)
{
    // 对当前选中行作删除标记
    int rowIndex = dataGridView1.CurrentRow.Index;
    dt.Rows[rowIndex].Delete();
    // 显示删除行的行状态
    dataGridView1.CurrentRow.Cells[0].Value
    =dt.Rows[rowIndex].RowState.ToString();
}

```

步骤六：选中某一行信息，如果需要修改该行的信息，则点击‘修改’按钮，则该行的 rowState 将显示 Modify，并且相应的信息也会随之改变，添加如下代码到主程序中：

```

private void button3_Click(object sender, EventArgs e)
{
    // 修改当前选中行
    int rowIndex = dataGridView1.CurrentRow.Index;
    DataRow dr = dt.Rows[rowIndex];
    dr[0] = txtPlace.Text;
    dr[1] = txtPname.Text;
    dr[2] = txtTname.Text;
    dr[3] = txtNumber.Text;
    dr[4] = txtGoals.Text;
    // 行状态及修改值显示到表格中
    DataGridViewRow row = dataGridView1.CurrentRow;
    row.Cells[0].Value = dr.RowState.ToString();
    row.Cells[1].Value = dr[0];
    row.Cells[2].Value = dr[1];
    row.Cells[3].Value = dr[2];
    row.Cells[4].Value = dr[3];
}

```

步骤七：以上所做的操作都是在关闭与数据库的连接时在缓存中的数据表格中做的操作，如果需要同步到数据库的信息中去，则必须调用 update 方法，所以点击按钮‘确定’，将所有的修改都同步到数据库中去，添加如下代码到主程序中去：

```

private void button4_Click(object sender, EventArgs e)
{

```

```

try
{
    SqlCommandBuilder builder = new SqlCommandBuilder(da);
    da.Update(ds, "GoalScore");
    FillGrid();
}
catch (SqlException ex)
{
    MessageBox.Show(ex.Message);
}
}

```

步骤八： 如果不想保存刚才进行的操作，则不必进行上面一步，即不必点击‘确定’按钮，而需要点击‘重置’按钮，则所有的操作将还原，这时将看到所有的 rowState 都恢复到初始的状态，添加如下代码到主程序中去：

```

private void button5_Click(object sender, EventArgs e)
{
    ds.RejectChanges();
    FillGrid();
}

```

至此，所有的该窗口的功能已经全部实现。